Innominatum CONSULTORIA

Gestão de Versões

ServiceNow - Deloitte

Prof. Tiago Rodrigues

IPCA POLITÉCNICO DO CÁVADO E DO AVE
INSTITUTO DO EMPREGO E FORMAÇÃO PROFISSIONAL
PRR Plano de Recuperação e Resiliência
REPÚBLICA PORTUGUESA
Financiado pela União Europeia NextGenerationEU
IAPMEI Parcerias para o Crescimento
PORTUGAL DIGITAL

Objetivo

Implementar estratégias avançadas de controlo de versões com Git e GitHub, aplicando modelos de branching, revisão estruturada de código e versionamento semântico, garantindo colaboração segura, rastreabilidade e qualidade no ciclo de desenvolvimento.

Objetivos Específicos

Ao final da sessão, os formandos deverão ser capazes de:

- Comparar e selecionar estratégias de branching (GitHub Flow, Git Flow e Trunk-Based) de acordo com o contexto do projeto;
- Configurar e gerir branches segundo o modelo GitHub Flow;
- Implementar branch protection rules garantindo revisão obrigatória de código;

Objetivos Específicos (cont.)

- Executar pull requests estruturados (incluindo draft PR e code review formal);
- Resolver conflitos de merge de forma autónoma;
- Aplicar Conventional Commits e versionamento semântico (SemVer);
- Criar e gerir tags e releases no GitHub;
- Configurar corretamente ficheiros .gitignore e .gitattributes para projetos Node/Angular.

Innominatum CONSULTORIA

Git Avançado e GitHub Flow

É útil para todos, não apenas para os desenvolvedores.

IPCA POLITÉCNICO DO CÁVADO E DO AVE
INSTITUTO DO EMPREGO E FORMAÇÃO PROFISSIONAL
PRR Plano de Recuperação e Resiliência
REPÚBLICA PORTUGUESA
Financiado pela União Europeia NextGenerationEU
IAPMEI Parcerias para o Crescimento
PORTUGAL DIGITAL

Revisão rápida: Repositório

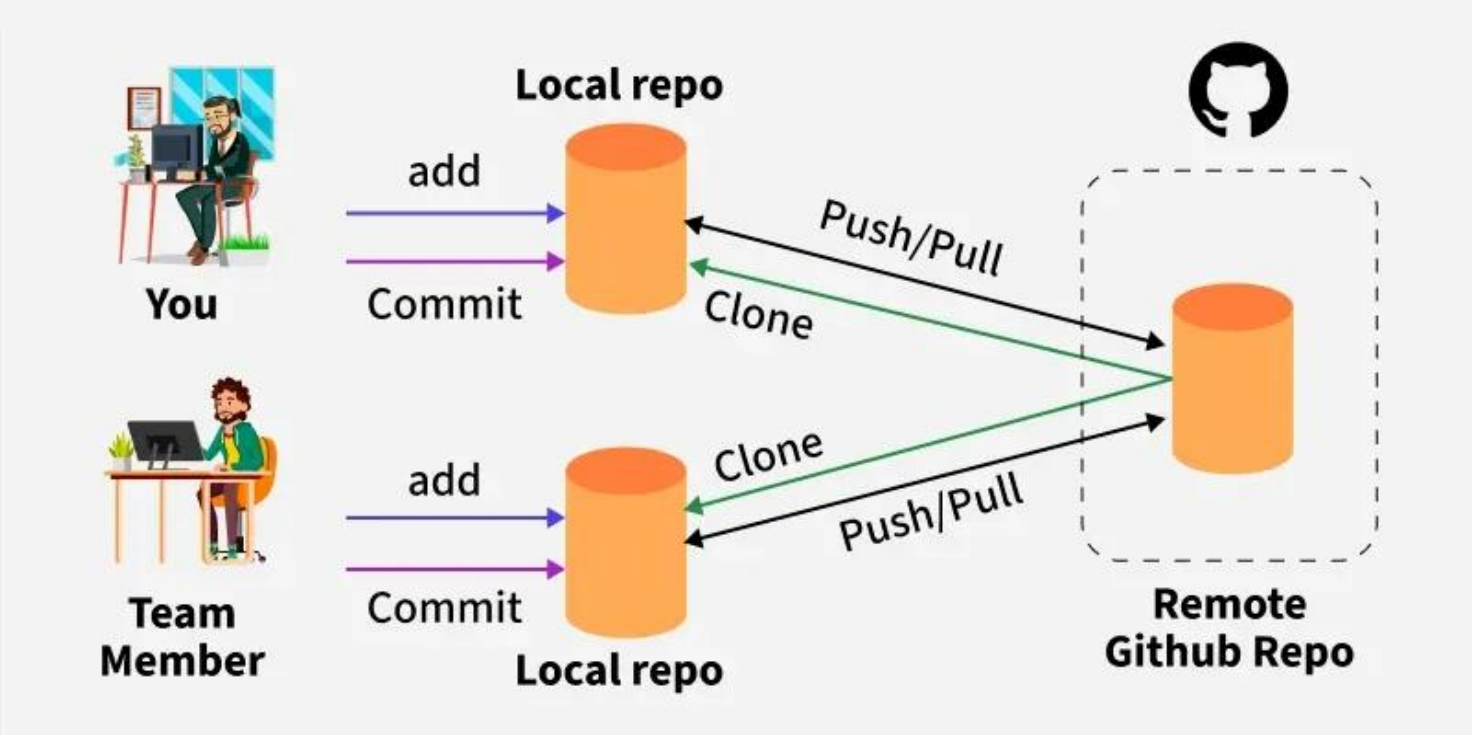
Um repositório Git (ou repo) é um **espaço de armazenamento** onde todos os **ficheiros** de um projeto e todo o seu **histórico** completo de **alterações** são mantidos. Permite **rever**, **gerir**, e **colaborar** no desenvolvimento ao longo do tempo.

Revisão rápida: Repositório (cont.)

- Guarda todos os **ficheiros**, **branches**, **commits**, e **histórico** de um projeto;
- Permite controlo de versão: é possível **reverter** o projeto para um estado anterior;
- Possibilita a **colaboração**: múltiplos desenvolvedores podem trabalhar em conjunto **sem sobreporem as alterações** feitas por cada elemento;
- Pode ser **clonado** para criar cópias em diferentes máquinas.

Revisão rápida: Repositório - Tipos

Pode ser **local** ou **remoto**.



Revisão rápida: Repositório local

- Guardado no computador;
- Permite alterar, registar, e rever o histórico do projeto sem necessidade de uma ligação à internet.

Revisão rápida: Repositório remoto

- Hospedado num servidor como **GitHub**, **GitLab**, ou **Bitbucket**;
- Possibilita a colaboração de vários desenvolvedores no mesmo projeto;
- Permite operações como ***push***, ***pull*** e ***fetch*** para sincronizar as alterações com o repositório local.

Revisão rápida: Commits

Registo de **alterações** feitas a um ou mais **ficheiros**, identificado por um **ID único**, que indica que alterações foram realizadas, **quando** e por **quem**.

Representa um estado específico do projeto num determinado momento.

Revisão rápida: Push/Pull

- Push
  - Faz o envio (upload) dos commits locais para um repositório remoto (ex:. Um repositório noutra máquina na mesma rede).
- Pull
  - Faz a transferência (download) das alterações de um repositório remoto e aplica-as ao branch local.

Estratégias de branching: GitHub Flow vs Git Flow vs Trunk-Based

Num mundo moderno de desenvolvimento de software, o controlo de versão é essencial, e o Git emergiu como o padrão da indústria.

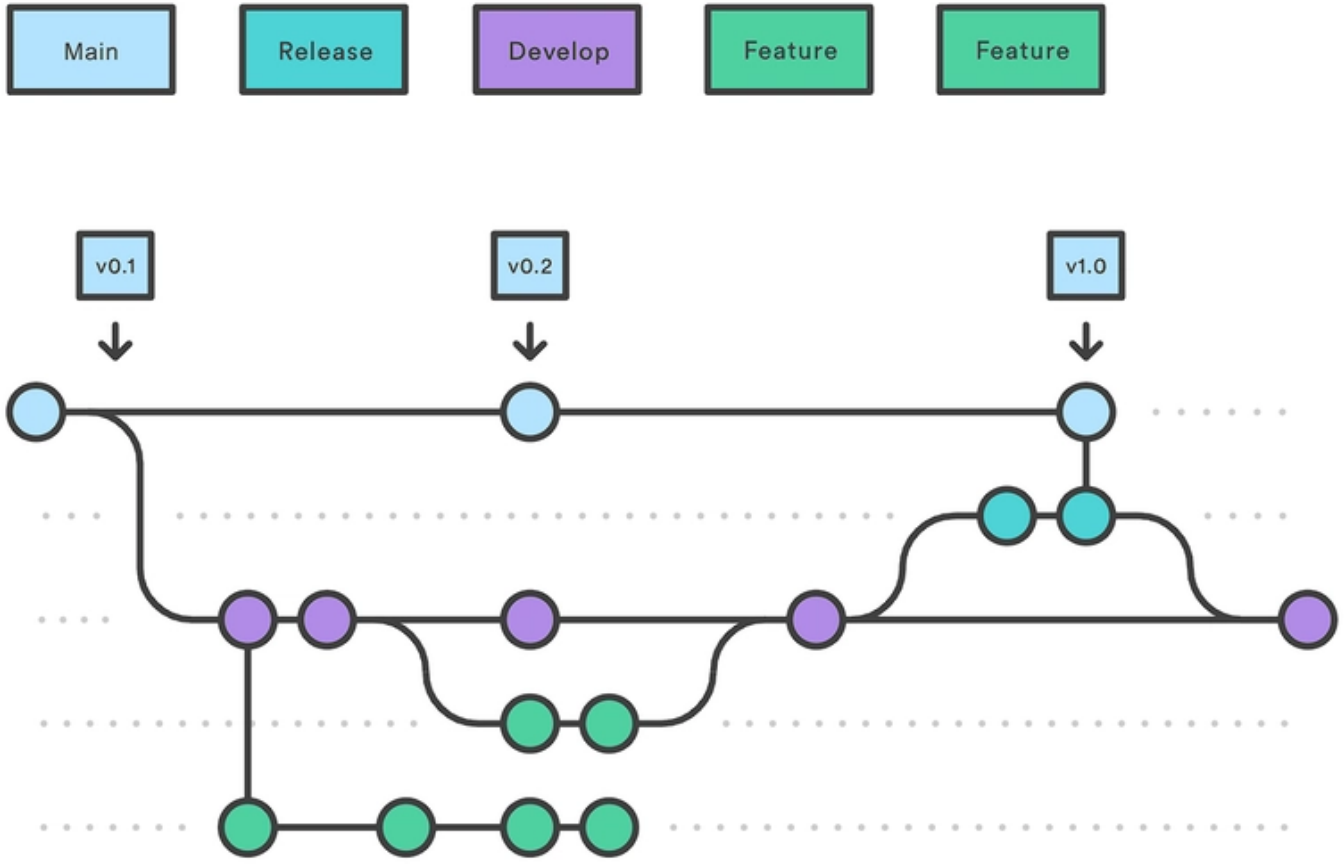
Contudo, o Git ,por si só, não é suficiente. É necessária uma estratégia de ramificação (branching) bem definida e alinhada com o fluxo de trabalho e com os requisitos da equipa e do projeto.

Estratégias de branching: Git Flow

GitFlow é um dos modelos de **ramificação** (branching) mais conhecidos.

Caracteriza-se por uma **estrutura bem definida**, concebida para **organizar** e **gerir** lançamentos de software.

Estratégias de branching: Git Flow



Estratégias de branching: Git Flow - Estrutura

- Branches principais:
  - main: Guarda o histórico oficial de lançamentos;
  - develop: Branch de integração para novas funcionalidades. É um ponto de integração contínua do trabalho da equipa.

- Branches de apoio:
  - feature/*: Desenvolvimento de nova funcionalidade;
  - release/*: Preparação de lançamentos;
  - hotfix/*: Para resolução de problemas críticos em ambiente de produção;
  - bugfix/*: Para corrigir erros identificados na branch *develop.*

Estratégias de branching: Git Flow - Workflow

1. O desenvolvimento ocorre na branch ***develop;***

2. O trabalho de novas funcionalidades é realizado em branches ***feature/*** , que são **branches locais** (na máquina do developer) criadas a partir da ***develop.*** Este padrão de nomes ajuda a rastrear os tipos de trabalho e facilita a recolha de métricas*;*

3. Quando um conjunto suficiente de funcionalidades está concluído, é criada uma branch ***release/;***

Estratégias de branching: Git Flow - Workflow

4. Quando está pronta, a branch ***release/*** é integrada tanto na *main* como na ***develop***;

5. Problemas críticos em produção são corrigidos em branches ***hotfix/;***

6. As ***hotfix/*** são integradas tanto na *main* como na ***develop***.

Estratégias de branching: Git Flow - Pros

- Estrutura bem definida e versionamento claro;
- Separa o desenvolvimento em curso e trabalho finalizado;
- Adequado para lançamentos planeados e processos formais de garantia de qualidade (QA);
- Permite o desenvolvimento simultâneo de múltiplas versões.

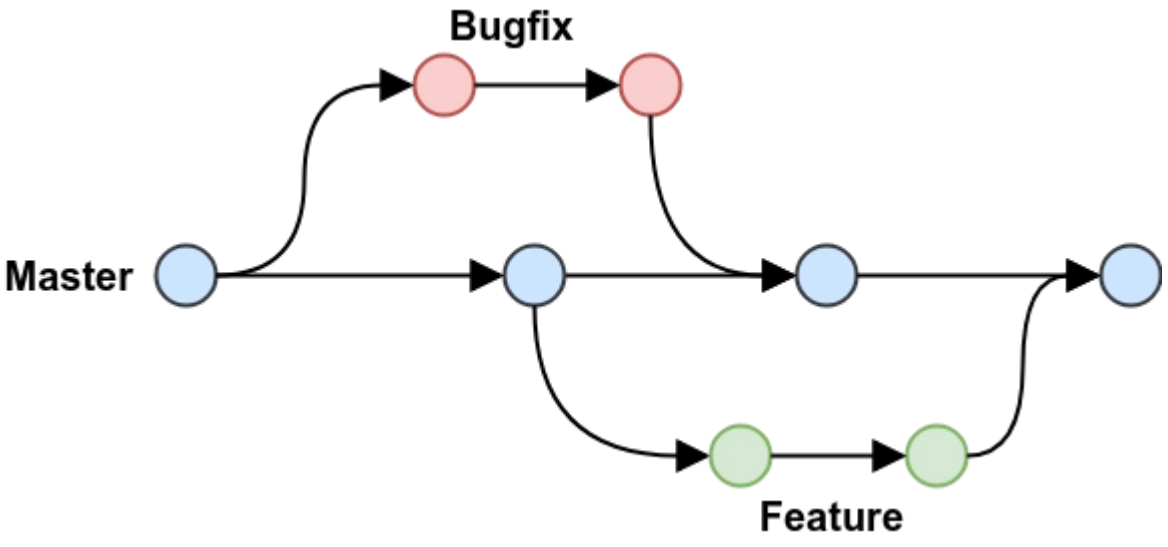
Estratégias de branching: Git Flow - Contras

- Estrutura complexa, com múltiplas ramificações a gerir;
- Sobrecarga desnecessária em projetos de pequena dimensão;
- Pouco adequado a práticas de *deployment* contínuo;
- Pode resultar em conflitos de integração significativos.

Estratégias de branching: GitHub Flow

O GitHub Flow é um modelo de fluxo de trabalho simples e baseado em ramificações, desenvolvido pelo GitHub. Apresenta menor complexidade comparativamente ao GitFlow e foi concebido para equipas que adotam práticas de entrega contínua *(continuous delivery - CD).*

Estratégias de branching: GitHub Flow



Estratégias de branching: GitHub Flow - Estrutura

- Branch principal:
  - main: Sempre pronto para produção com código estável.

- Branches de apoio:
  - feature/bugfix branches: Criadas a partir da *main* e posteriormente reintegradas (merge) com na *main.*

Estratégias de branching: GitHub Flow - Workflow

1. Criação de uma branch a partir da main para desenvolvimento;
2. Realização de commits nessa branch;
3. Abertura de um pull request para revisão;
4. Revisão, testes e eventuais ajustes;
5. Integração (merge) na main após aprovado;
6. Colocação imediata em produção após integração.

Estratégias de branching: GitHub Flow - Pros

- Simples e de fácil compreensão;
- Apropriado para entrega contínua;
- Minimiza a gestão de ramificações (branches);
- Foca-se na colaboração por *pull request;*

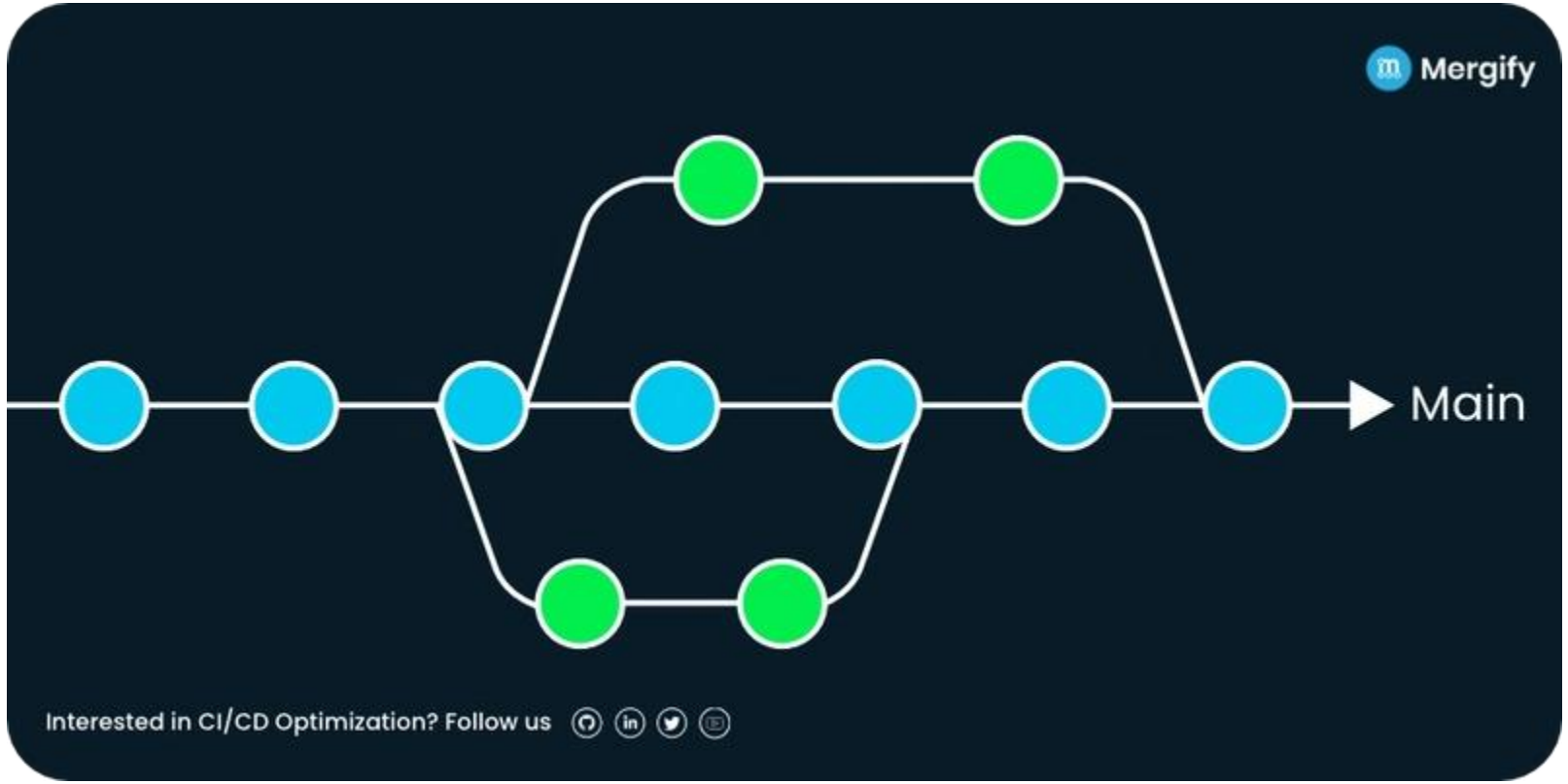
Estratégias de branching: GitHub Flow - Contras

- Capacidade limitada para gestão estruturada de lançamentos;
- Não define claramente uma fase intermedia de testes ou integração;
- Dependência significativa de testes automatizados robustos para mitigar riscos;
- Pouco adequado à manutenção simultânea de várias versões do software.

Estratégias de branching: Trunk-Based

O desenvolvimento **Trunk-Based** (tronco) é um **modelo de controlo de versão** no qual os desenvolvedores trabalham predominantemente numa **única *branch* principal**, designada por ***trunk*** (geralmente *main*), **evitando** *branches* de **funcionalidades prolongadas**.

Estratégias de branching: Trunk-Based



Estratégias de branching: Trunk-Based - Estrutura

- Main branch:
  - main: Representa a fonte única de verdade, concentrando todo o desenvolvimento.
- Branches de apoio:
  - Branches de funcionalidades de curta duração (normalmente com duração máxima de 1 a 2 dias);
  - Branches de lançamento (opcionais, destinadas à estabilização da versão antes da sua disponibilização).

Estratégias de branching: Trunk-Based - Workflow

1. Realização de **commits pequenos** e **frequentes** diretamente na ***main;***

2. Utilização de ***feature toggles*** para controlar a **disponibilização** de **funcionalidades incompletas**;

3. **Integração diária** de branches de curta duração na ***main;***

4. Recurso a práticas de **Integração Contínua e Entrega Contínua** (*Continuous Integration / Continuous Delivery – CI/CD)* para assegurar a **estabilidade** do ***trunk;***

5. Criação opcional de **branches de lançamento** para **estabilização** antes da **disponibilização**.

Estratégias de branching: Trunk-Based - Pros

- Impõe a prática de integração contínua;
- Minimiza conflitos de integração (*merge conflicts*);
- Permite uma verdadeira entrega contínua (*continuous delivery*);
- Incentiva alterações pequenas e incrementais.

Estratégias de branching: Trunk-Based - Contras

- Requer uma infraestrutura de testes sofisticada e robusta;
- Depende da utilização de *feature toggles;*
- Menor isolamento entre funcionalidades;
- Maior risco de introdução de erros na *main.*

Estratégias de branching: Síntese

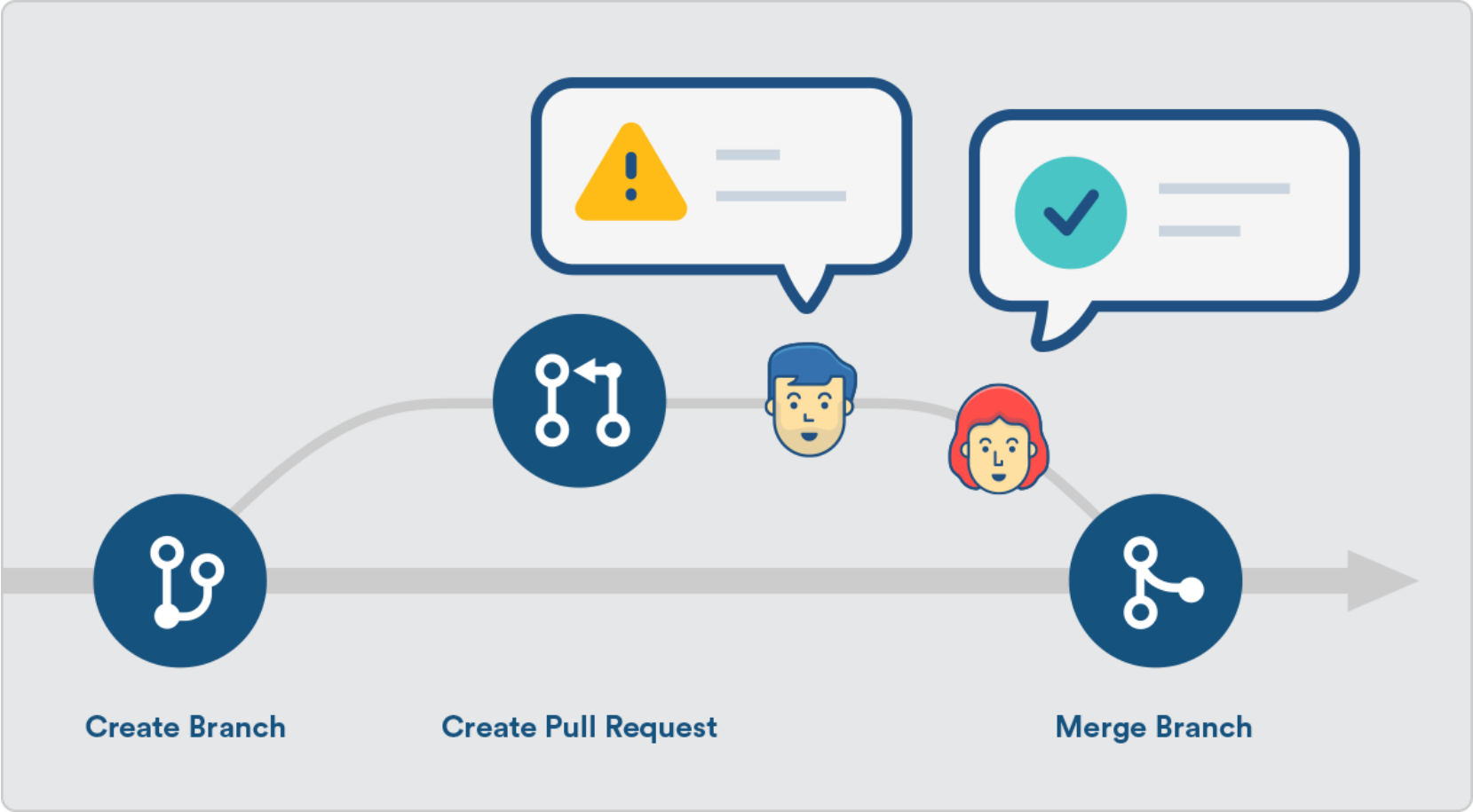
Critério	GitFlow	GitHub Flow	Trunk-Based Development
Complexidade	Elevada	Baixa	Moderada
Branch principal	`master/main` + `develop`	`main`	`main` (trunk)
Branches de suporte	`feature/*`, `release/*`, `hotfix/*`, `bugfix/*`	`feature/bugfix`	Feature branches curtas (opcionais)
Duração das branches	Média a longa	Curta	Muito curta (1–2 dias)
Gestão de lançamentos	Estruturada e formal	Simples	Opcional
Adequação a CI/CD	Menos adequado a deployment contínuo	Adequado a entrega contínua	Fortemente orientado a CI/CD
Risco de conflitos	Pode ser elevado	Moderado	Reduzido (integração frequente)
Adequado para	Projetos com versões bem definidas e ciclos de release formais	Equipas pequenas/médias com entrega contínua	Equipas maduras com forte automação e integração contínua

Estratégias de branching: Consolidação

Quando usar cada um dos seguintes branches:

- feature/*: Quando se pretende isolar o desenvolvimento de novas funcionalidades do ambiente produtivo;
- hotfix/*: Quando há um incidente crítico já em produção e é necessário corrigir com impacto mínimo;
- release/*: Quando é necessário um período de estabilização (QA, ajustes de configuração, versionamento).

Pull Requests: anatomia, draft PR, code review estruturado



Pull Requests: O que é?

Uma **pull request (PR)** é um **pedido formal** para **integrar alterações** de uma branch noutra (normalmente a *main* ou *develop).*

Funciona como ponto central de **revisão de código**, **discusssão técnica** e **validação automática** (CI), **antes da integração**.

Pull Requests: Anatomia

- Título informativo;
- Contexto bem definido;
- Escopo bem definido;
- Mensagens de commit claras e informativas.

Pull Requests: Anatomia - Título

- Evitar:
  - “Updated code”;
  - “Fixed bug”;
  - “changes from yesterday.
- Boas práticas:
  - feat: Add OAuth2 authentication for API endpoints;
  - fix: Resolve race condition in user session handling;
  - refactor: Optimize database query performance in user search.

Pull Requests: Anatomia - Contexto

```
## Problema

Os utilizadores não conseguem repor a palavra-passe quando se esquecem dela.
Neste momento, têm de contactar o suporte, o que é lento e frustrante.

Ticket relacionado: AUTH-456

## Solução

Foi adicionada a funcionalidade "Esqueci-me da palavra-passe".

* Adicionado um link "Esqueci-me da palavra-passe?" na página de login
* Criado um formulário onde o utilizador pode inserir o seu email
* Enviado um link de reposição para o email do utilizador
* Permitido ao utilizador criar uma nova palavra-passe através desse link

## Detalhes Técnicos

* Adicionada uma nova página `/forgot-password`
* Criado um endpoint da API `/reset-password`
* Adicionado um campo `resetToken` na base de dados do utilizador
* Os tokens expiram ao fim de 1 hora
```

```
## Testes

1. Ir à página de login
2. Clicar em "Esqueci-me da palavra-passe?"
3. Inserir um email válido
4. Abrir o email recebido e clicar no link de reposição
5. Criar uma nova palavra-passe
6. Iniciar sessão com a nova palavra-passe

Resultado esperado: O utilizador consegue iniciar sessão com a nova palavra-passe.

## Configuração

Novas variáveis de ambiente necessárias:

* EMAIL_API_KEY
* FRONTEND_URL
```

Innominatum CONSULTORIA
IPCA Politécnico do Cávado e do Ave · Instituto do Emprego e Formação Profissional · PRR Plano de Recuperação e Resiliência · República Portuguesa · Financiado pela União Europeia NextGenerationEU · IAPMEI Parcerias para o Crescimento · Portugal Digital

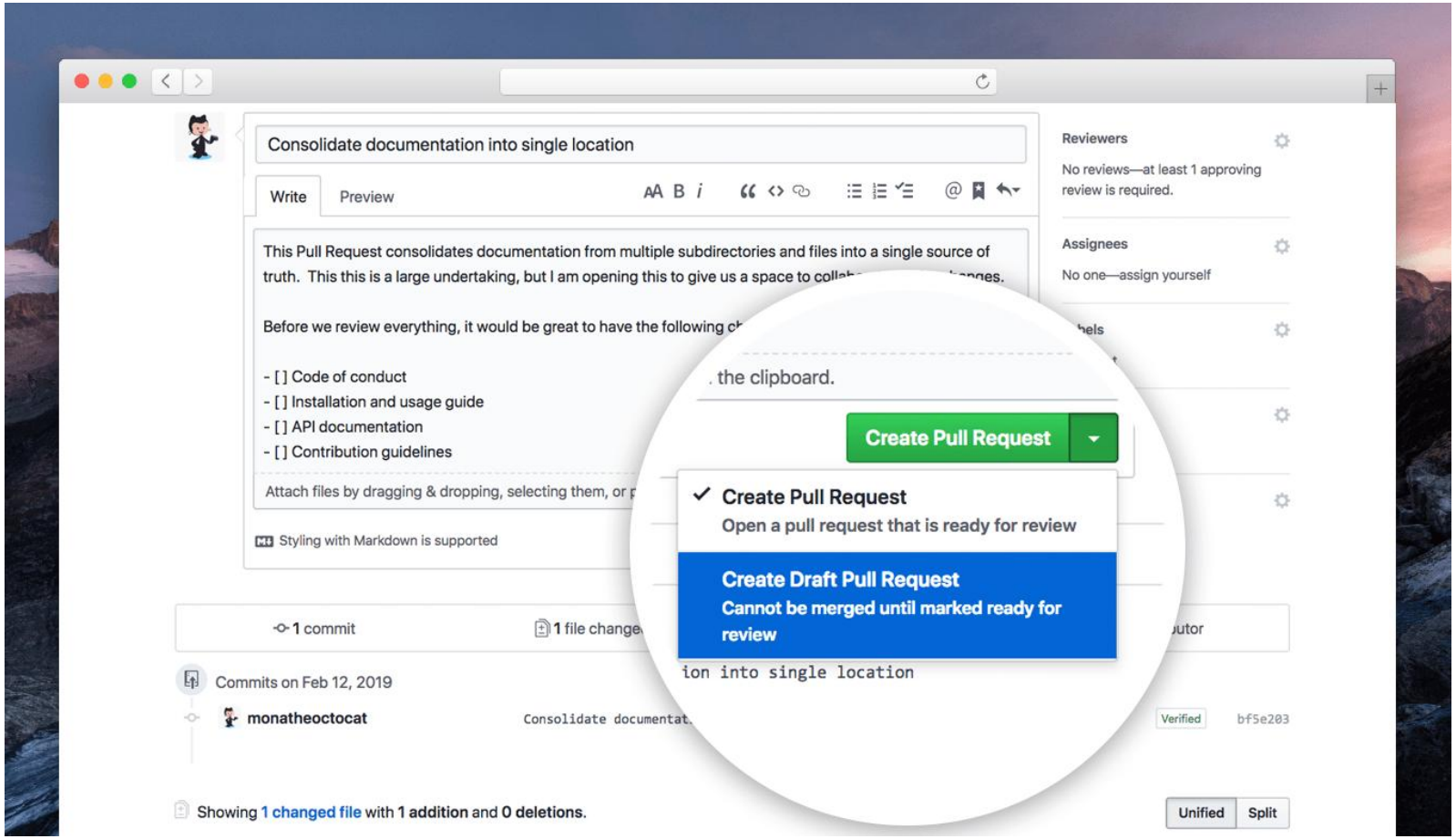
Pull Requests: Anatomia - Escopo

- Primeiro PR: feat: Add basic user model and migration;
- Segundo PR: feat: Implement user authentication endpoints;
- Terceiro PR: feat: Add user profile management UI;
- Fourth PR: feat: Integrate email verification system;

Pull Requests: Anatomia – Mensagens

- Mensagens de commit claras e informativas
  - Má prática: "Fixed stuff“;
  - Boa prática: fix(auth): resolve user login bug caused by expired tokens.

Pull Requests: Draft PR



Pull Requests: Draft PR – O que é?

Uma draft pull é essencialmente um pull request em **estado preliminar**, utilizada para partilhar **trabalho em progresso** (WIP) sem iniciar formalmente o processo de revisão e integração.

Enquanto estiver em modo *draft*, a PR **não pode ser integrada** (merge) e não solicita automaticamente a revisão aos responsáveis pela mesma.

Pull Requests: Draft PR – O que é?

- Não pode ser integrado;
- Trabalho em progresso (WIP);
- Não solicita revisão automática;
- Pode ser marcada como ``Ready for Review'' para obter feedback.

Pull Requests: Code Review Estruturado

Qualidade do código

- O código é legível?
- Segue os padrões do projeto?
- Existem duplicações?

Pull Requests: Code Review Estruturado – O que é?

Code review estruturado é uma **revisão de código feita com critérios e processos definidos**, em vez de uma revisão informal ou superficial. O objetivo é garantir **qualidade**, **consistência e segurança do código** antes de ele ser integrado na branch principal.

Pull Requests: Code Review Estruturado – O que é?

Em contexto de **GitHub / Pull Requests**, significa que a revisão segue **um conjunto de passos e regras claras**.

O autor do Pull Request deve explicar claramente:

- O que foi alterado;
- Porquê a alteração foi necessária;
- Como testar a alteração.

Pull Requests: Code Review Estruturado – Checklist

O revisor segue critérios consistentes, por exemplo:

- Qualidade do código;
- Arquitetura;
- Boas práticas;
- Segurança;
- Performance.

Innominatum CONSULTORIA
IPCA POLITÉCNICO DO CÁVADO E DO AVE
INSTITUTO DO EMPREGO E FORMAÇÃO PROFISSIONAL
PRR Plano de Recuperação e Resiliência
REPÚBLICA PORTUGUESA
Financiado pela União Europeia NextGenerationEU
IAPMEI Parcerias para o Crescimento
PORTUGAL DIGITAL

Pull Requests: Code Review Estruturado – Checklist

Arquitetura

- A solução é adequada?
- O código está bem modulorizado?

Pull Requests: Code Review Estruturado – Checklist

Boas práticas

- Naming consistente;
- Funções pequenas e claras;
- Tratamento de erros.

Pull Requests: Code Review Estruturado – Checklist

Segurança

- Não credenciais expostas?
- Inputs estão validados?

Pull Requests: Code Review Estruturado – Checklist

Performance

- Existem operações desnecessárias?

Branch protection rules no GitHub

As **regras de proteção de branches** definem se os colaboradores podem eliminar ou fazer *force push* para uma branch e estabelecem **requisitos para qualquer envio de alterações (push)**, como a obrigatoriedade de passar **verificações de estado (*status checks*)** ou manter um **histórico de commits linear**.

Branch protection rules no GitHub - Exemplo

Protect matching branches

- [x] **Require a pull request before merging**
  When enabled, all commits must be made to a non-protected branch and submitted via a pull request before they can be merged into a branch that matches this rule.
  - [x] **Require approvals**
    When enabled, pull requests targeting a matching branch require a number of approvals and no changes requested before they can be merged.
    Required number of approvals before merging: 1
  - [ ] **Dismiss stale pull request approvals when new commits are pushed**
    New reviewable commits pushed to a matching branch will dismiss pull request review approvals.
  - [ ] **Require review from Code Owners**
    Require an approved review in pull requests including files with a designated code owner.
  - [ ] **Require approval of the most recent reviewable push**
    Whether the most recent reviewable push must be approved by someone other than the person who pushed it.

Conventional Commits

```
git commit -m "test: add test for create product automation"
git commit -m "feat: implement tracking product service"
git commit -m "refactor: change return log pattern"
git commit -m "style: change function param for objects"
git commit -m "fix: remove getPayment() wrong attribute"
git commit -m "chore: add no-undef rule in eslintrc.json"
git commit -m "docs: add technologies list in readme"
git commit -m "style: remove all blank spaces"
git commit -m "perf: change looping for parallel execution"
git commit -m "build: remove moment.js dependency"
git commit -m "build: add date-fns npm dependency"
git commit -m "revert: back to a215868 commit"
```

Conventional Commits: General Commit

```
<tipo>(<escopo opcional>): <descrição>
linha vazia como separador
<corpo opcional>
linha vazia como separador
<rodapé opcional>
```

Conventional Commits – Initial, Merge e Revert Commit

Initial Commit

```
Chore: init
```

Merge Commit

```
Merge branch ‘<nome da branch>’
```

Revert Commit

```
Revert “<mensagem do commit revertido>”
```

Conventional Commits: Tipos

- Alterações relevantes à API ou UI:
  - feat: **adicionam**, **ajustam** ou **removem** alguma funcionalidade;
  - fix: **resolvem** de problemas/**bugs** de um feat commit precedente.
- refactor: **rescrevem** ou **restruturam** código **sem alterar** o comportamento;
  - perf :tipo especial de refactor commits que **melhoram** especificamente o **performance.**
- style: referente ao **estilo de estruturação de código** (ex: espaços, formatação, falta de ponto-e-vírgulas) e **não afetam o comportamento da aplicação;**

Conventional Commits: Tipos (cont.)

- test: **acrescentam testes** em falta **ou corrigem** os já existentes;
- docs: afetam exclusivamente documentação;
- build: afetam componentes relacionados com o processo de build, como ferramentas de compilação, dependências, versão do projeto, entre outros;

Conventional Commits: Tipos (cont.)

- ops: afetam aspetos operacionais como insfraestrutura (IaC), scripts de deployment, pipelines CI/CD, backups, monitorização, ou procedimentos de recuperação,...
- chore: representam tarefas como commit inicial, modificar .gitignore, ...

Conventional Commits: Exemplos

```
feat: add email notifications on new direct messages
```

```
feat(shopping cart): add the amazing button
```

```
fix: add missing parameter to servisse cal


The error occurred due to <reasons>
```

```
refactor: implemente fibonacci number calculation as recursion
```

SemVer (Semantic Versioning)

Formato: **MAJOR.MINOR.PATCH**

- **MAJOR** – Alterações incompatíveis (*breaking changes*);
- **MINOR** – Novas funcionalidades compatíveis;
- **PATCH** – Correções de erros compatíveis.

Extensões:

- -alfa, -beta, -rc -> versões de pré-lançamento.

semver.org

Tags e Releases

Add file | Code

inutes ago | 7,049 Commits

	last year
	1 hour ago
	last week
	7 months ago
(#8209)	2 months ago
licate rem...	2 months ago
	3 months ago
#8392)	19 hours ago
	2 months ago
sion (#841...	19 hours ago
	7 months ago
	2 years ago

About

GitHub's official command line tool

cli.github.com

git | cli | golang | github-api-v4

- Readme
- MIT license
- Code of conduct
- Security policy
- Activity
- **33.9k** stars
- **858** watching
- **5.7k** forks

Report repository

Releases 140

GitHub CLI 2.39.2 Latest
last week

+ 139 releases

Tags e Releases

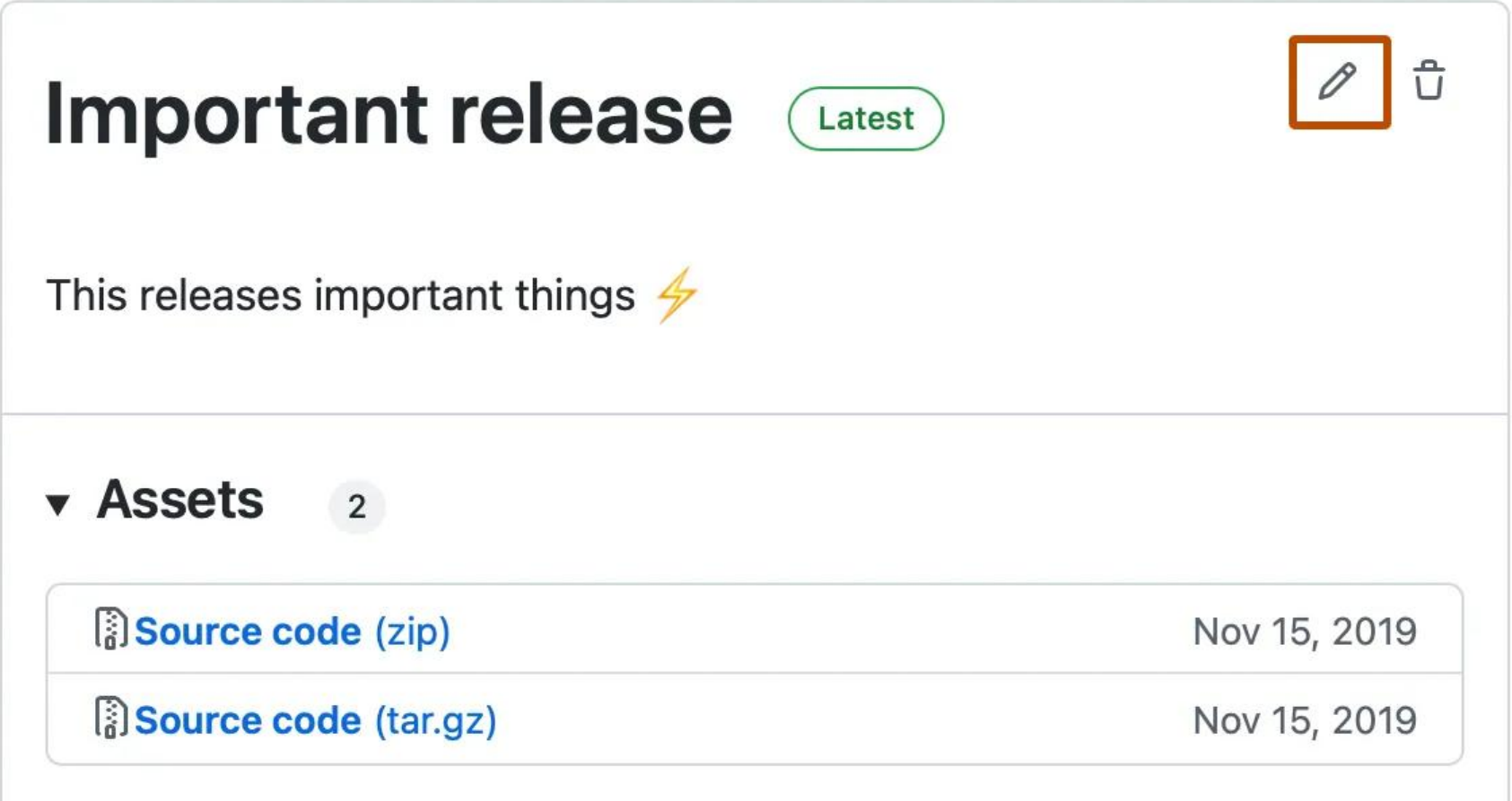
Releases são versões publicadas do software, preparadas para distribuição e utilização.

- Baseiam-se em **Git tags** (marcam um ponto específico no histórico);
- Permitem disponibilizar artefactos (ZIP, tar.gz) para download;

Tags e Releases

- Apenas utilizadores com **permissões de escrita** podem gerir releases;
- Podem incluir **notas de versão** (manuais ou geradas automaticamente);
- Podem acionar **avisos de segurança** e alertas (ex.: Dependabot);
- A informação pode ser consultada via **Releases API**.

Tags e Releases



Tags e Releases: Tag vs Release – Git Tag

- Marca um ponto específico no histórico do repositório;
- Identifica uma versão (ex: `v1.2.0`);
- É apenas uma referência no Git;
- Não implica publicação nem artefactos.

Tags e Releases: Tag vs Release – Release

- Baseada numa **tag**;
- Representa uma versão publicada do software;
- Pode incluir:
  - Notas de versão;
  - Ficheiros compilados (artefactos);
  - Informação de segurança;
- Destina-se a distribuição.

.gitignore

O **.gitignore** é um **ficheiro de configuração** utilizado pelo Git para especificar quais os ficheiros ou diretórios não devem ser incluídos no **controlo de versão**. É especialmente útil para evitar o **versionamento** de ficheiros temporários, artefactos de compilação, configurações locais ou informação sensível.

.gitignore: Exemplo

```
</> gitignore


node_modules/
dist/
coverage/
.env
.vscode/
*.log
.DS_Store
```

.gitattributes

O .gitattributes é um **ficheiro de configuração que** define regras específicas sobre como o Git deve tartar determinados ficheiros dentro do repositório.

Permite controlar aspetos como **normalização de line endings, estratégias de merge** e comportamento de diff, garantindo **consistência**entre diferentes sitemas operativos e ambientes de trabalho.

.gitattributes: Exemplo

```
</> gitattributes

# Normalizar finais de linha automaticamente
# Ex: CRLF (windows) / LF (Linux)
* text=auto

# Tratar imagens como binárias (sem merge de texto)
*.png binary
*.jpg binary
```

Innominatum CONSULTORIA

Gestão de Versões

ServiceNow - Deloitte

Prof. Tiago Rodrigues

IPCA POLITÉCNICO DO CÁVADO E DO AVE
INSTITUTO DO EMPREGO E FORMAÇÃO PROFISSIONAL
PRR Plano de Recuperação e Resiliência
REPÚBLICA PORTUGUESA
Financiado pela União Europeia NextGenerationEU
IAPMEI Parcerias para o Crescimento
PORTUGAL DIGITAL